

FFT-512 Acceleration on CV32A6 Using Custom ISA Extensions

Younes Benamara
Paris Saclay University

Charbel Ghanem
Paris Saclay University

Gautier Arson
Paris Saclay University

Mathieu Oriol
Polytechnic Institute of Paris

Abstract—This report presents an FFT-512 acceleration for the CV32A6 (CVA6, RV32IM_Zicsr) core. The approach adds custom instructions for packed complex arithmetic, radix-2 and radix-4 butterfly operations, and an optional twiddle register file, while keeping the original pipeline structure intact. We detail the ISA, micro-architectural integration, and software scheduling that map radix-4 butterflies to a controlled BFY4 protocol with Q15 scaling and bit-exact output. We report instruction and cycle counts before and after optimization, FPGA timing and utilization, and functional validation for FFT.

I. INTRODUCTION

The 2025–2026 contest targets the acceleration of a 512-point fixed-point FFT using a CV32A6-based platform. The baseline application uses kissFFT and must remain bit-exact after optimization. The key constraints are compatibility with RV32IM_Zicsr, no coarse-grain FFT hardware accelerator, no cache size increase, and no more than 20% frequency degradation. Our solution focuses on custom ISA extensions integrated into the existing decode, ALU, and MULT units.

II. DESIGN OVERVIEW

We implement a tiered ISA strategy:

- **Tier 1: Packed complex ALU** with CADD16, CSUB16, CMUL16, CMADD16, CMSUB16.
- **Tier 2: BFY2** (radix-2 butterfly) with BFY2, BFY2H.
- **Tier 3: BFY4** (radix-4 butterfly) with staged setup BFY4_S0/S1/S2 and readout BFY4_O0..O3.
- **Tier 4: Twiddle register file** controlled by TWLD and TWCFG.

The BFY4 unit is stateful: three setup steps load inputs and twiddles, then four explicit output reads return the radix-4 results. This keeps software control while avoiding a hardwired FFT block.

III. INSTRUCTION SET EXTENSIONS AND DATA REPRESENTATION

Packed complex format: each complex sample is stored in one 32-bit register, with real in bits [15:0] and imag in [31:16], using Q15 fixed-point. This representation halves register pressure and aligns naturally with 32-bit loads and stores. For complex multiply, intermediate 32-bit products are rounded with $q15(x) = \lfloor (x + 2^{14}) / 2^{15} \rfloor$ to match the reference software behavior.

Opcode allocation and rs3 usage: Custom0 encodes packed ALU ops and twiddle control (TWLD/TWCFG),

Custom1 encodes BFY2, and Custom2 encodes BFY4. Three-operand operations reuse the rs3 read path, allowing fused mul-add/mul-sub and BFY2/BFY4 to consume twiddle operands without extra moves.

TABLE I: Summary of custom instructions.

Opcode	Mnemonic	Role
Custom0 (0x0B)	CADD16/CSUB16/CMUL16	Packed complex ALU ops
Custom0 (0x0B)	CMADD16/CMSUB16	Fused complex mul-add/sub
Custom0 (0x0B)	TWLD/TWCFG	Twiddle RF load/config
Custom1 (0x2B)	BFY2/BFY2H	Radix-2 butterfly outputs
Custom2 (0x5B)	BFY4_S0/S1/S2	Radix-4 setup/compute
Custom2 (0x5B)	BFY4_O0..O3	Radix-4 output reads

IV. HARDWARE MODIFICATIONS

Custom opcodes Custom0/1/2 are decoded in `decoder.sv`. Packed complex operations are implemented in `alu.sv`. The MULT unit contains BFY2/BFY4 logic and an optional twiddle register file. No new pipeline stage is added, and the standard register file is reused (including rs3 for 3-operand operations).

Modified RTL files:

- `core/include/ariane_pkg.sv`: adds `fu_op` enums and `is_rs3_gpr()` for 3-operand instructions.
- `core/decoder.sv`: decodes Custom0/1/2 and routes to ALU/MULT.
- `core/alu.sv`: packed complex add/sub/mul and fused ops.
- `core/mult.sv`: BFY2/BFY4 pipelines and twiddle register file.

Packed complex ALU: the ALU computes add, sub, mul, and fused mul-add/mul-sub in a combinational datapath. Complex multiply uses four 16x16 products, forms the real and imag components, applies Q15 rounding, and packs the result back into a 32-bit word. This keeps the latency at one cycle and avoids structural hazards.

BFY2 pipeline: BFY2 is a two-step protocol. The first instruction captures inputs and computes the twiddle products; the second cycle produces the low output, while the high output is saved and read with BFY2H. This separates arithmetic from output access and keeps the pipeline simple.

BFY4 state machine: BFY4 uses three setup instructions to capture {a,b,c,d} and twiddles, then computes the radix-4 butterfly in the MULT unit. Outputs are stored in internal

registers and read explicitly through BFY4_O0..O3. This protocol avoids a hardwired FFT sequencer while still collapsing multiple complex operations into a few instructions.

BFY4 compute block: the BFY4_S2 path performs three complex multiplies, Q15 rounding, butterfly sums, and the final $\pm j$ rotations to produce o0..o3 in one cycle. The outputs are latched and made available through BFY4_O* reads, enabling software-controlled ordering without extra control hardware.

Twiddle register file: a 512-entry 32-bit twiddle register is controlled by TWLD/TWCFG. When enabled, the cache supplies twiddle values by index to BFY2/BFY4, reducing repeated memory loads. When disabled, the immediate path is used to preserve baseline behavior.

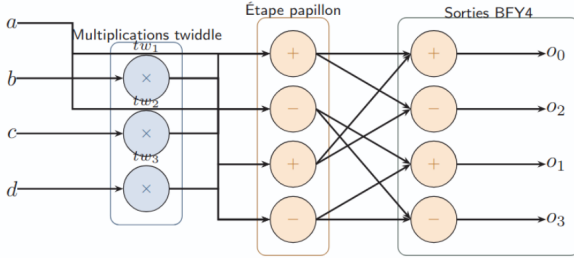


Fig. 1: Functional schema of the BFY4 protocol (S0/S1/S2 then O0..O3).

Output arbitration: the MULT unit prioritizes BFY4 results, then BFY2, then standard MUL/DIV results. This ensures BFY4 readbacks are not delayed by unrelated multiplier activity.

V. SOFTWARE INTEGRATION

The kissFFT stack is modified to call the custom instructions. Key changes include:

- Intrinsics and wrappers for custom ISA operations in `_kiss_fft_guts.h`.
- BFY2/BFY4 scheduling in `kiss_fft.c`.
- Runtime twiddle preload via TWLD/TWCFG in `fft_int16_main.c`.

The optimized FFT path keeps Q15 scaling consistent with the baseline and preserves bit-exact output.

Modified software files:

- `sw/app/fft/_kiss_fft_guts.h`: inline `.insn` intrinsics and packed load/store helpers.
- `sw/app/fft/kiss_fft.c`: BFY2/BFY4 hot paths and BFY4 protocol.
- `sw/app/fft/fft_int16_main.c`: twiddle preload with TWLD/TWCFG.
- `sw/app/Makefile`: flags to enable ISA tiers.

VI. SOFTWARE SCHEDULING AND ALGORITHM MAPPING

The FFT-512 implementation uses a radix-4 decomposition with a software-driven schedule. Each stage maps groups of butterflies to BFY4 where possible, falling back to BFY2 for

boundary cases. A “flat” schedule is used for the 512-point case to avoid runtime planning overhead and to maximize reuse of twiddle values. This scheduling strategy reduces loop overhead and keeps the instruction stream regular, which is important for an in-order core.

The flat path for 512 points includes digit-reversal, a fused stage-1/stage-2 schedule, and a sequence of radix-4 stages that match the twiddle reuse pattern. This avoids recursive scheduling overhead and allows the compiler to keep hot loops small and predictable.

Twiddle factors are preloaded into the twiddle register file at program start. The software then issues BFY4 setup instructions with twiddle indices, allowing the MULT unit to resolve twiddle operands internally. This removes repeated loads from memory and reduces register pressure. The fixed-point scaling is maintained throughout the pipeline so that the final output remains bit-exact with the baseline kissFFT reference.

VII. VERIFICATION AND COMPLIANCE

Functional correctness is checked by comparing the FFT output against a known golden vector using a bitwise comparison. The design keeps the RV32IM_Zicsr baseline intact, does not increase cache sizes, and does not introduce a coarse-grain FFT accelerator.

Tier enable flags: software compilation uses flags that enable each ISA tier. For example, `KISSFFT_USE_IM_CUSTOM` enables packed complex ops, `KISSFFT_USE_BFY2` enables BFY2, `KISSFFT_USE_BFY4` enables BFY4, and `KISSFFT_USE_TWIDDLE_RF` enables the twiddle register. An optional `KISSFFT_USE_BFY4_SCALE` flag fuses the $\div 4$ scaling into BFY4 inputs.

VIII. EXPERIMENTAL SETUP

Measurements are obtained for Zybo Z7-20 execution. The FFT console output provides instruction and cycle counts. Timing and resource utilization come from the post-place-and-route reports in the FPGA flow. **CoreMark and MNIST results are currently not available** because these binaries were not executed.

The FPGA reports were generated with Vivado 2024.1. Timing is taken from `cva6_fpga.timing.rpt` (setup path with positive slack), and utilization from `cva6_fpga.utilization.rpt` (top-level totals). The maximum frequency is computed as $f_{max} = 1/(T - slack)$ using the clock period defined in the timing report.

IX. RESULTS

A. FFT Performance

The expected trend from the tiered ISA is a monotonic reduction in instruction count as packed ops, BFY2, BFY4, and twiddle caching are enabled. The exact numbers are platform dependent; the tables below capture the measured values Zybo execution.

TABLE II: FFT-512 performance on Zybo Z7-20.

Version	Instructions	Cycles	Change (%)
Baseline	124143	164674	–
Optimized	29980	45585	-72,3%

(a) Baseline FFT console output.

(b) Optimized FFT console output.

Fig. 2: Screenshots of FFT results before and after optimization.

B. Timing and Resource Utilization

The post-P&R timing report shows a 25.000 ns clock period with 0.874 ns positive slack, corresponding to a theoretical f_{max} of 41.4 MHz for the optimized design. Resource utilization is reported at the top level and includes 60 RAMB36 blocks and 32 DSP blocks.

TABLE III: FPGA timing and utilization (post-P&R).

Version	Fmax (MHz)	LUTs	FFs	BRAM36	DSPs
Baseline	40.7	19804	14624	60	4
Optimized	41.4	25895	31482	60	32
Change (%)	+1.7	+30.8	+115.3	0.0	+700.0

X. DISCUSSION

The tiered ISA extensions reduce the instruction count and the number of cycles by fusing complex arithmetic and butterfly operations into single instructions. The BFY4 protocol trades a small amount of state for large software-side savings without introducing a hardwired FFT accelerator. The twiddle register file removes repeated loads and improves locality. The current report highlights the absence of CoreMark and MNIST runs.

XI. CONCLUSION

We presented a FFT-512 acceleration for CV32A6 using custom ISA extensions and software integration in kissFFT. The report includes metrics for FFT performance, FPGA timing and utilization.